



Universidad de Extremadura

Escuela Politécnica

Grado en Ingeniería Informática del Software

Mapecto de SpecKit contra flujos profesionales de ingeniería de requisitos basados en normas

Trabajo de Fin de Grado

Autor: Abdel Wahed Mahfoud Mouhandizi

Tutor: [Nombre del tutor] Curso 2025–2026

Resumen

Este trabajo presenta un mapeo sistemático del flujo de trabajo de **SpecKit**, un toolkit de código abierto para *Spec-Driven Development* desarrollado por GitHub, contra un flujo profesional de ingeniería de requisitos basado en normas. El marco normativo de referencia se compone del *IREB CPRE Foundation Level Handbook*, la norma ISO/IEC/IEEE 29148 y la *INCOSE Guide for Writing Requirements*.

El estudio documenta el flujo real de **SpecKit** (sus comandos, artefactos y mecanismos de calidad) a partir de sus fuentes oficiales, y lo contrasta con las actividades fundamentales de la ingeniería de requisitos —elicitación, documentación, validación y gestión— así como con los criterios de calidad definidos por las normas de referencia. Para fundamentar el mapeo con evidencia empírica, se construye un corpus de requisitos reales extraídos de *changelogs* de seis proyectos *open source* representativos (Appwrite, Authentik, Cal.com, Directus, Medusa y n8n), se formalizan siguiendo una plantilla con atributos IREB, y se introducen en **SpecKit** para observar el comportamiento del pipeline *Spec-Driven Development*.

El resultado principal es un mapa de alineación que identifica, para cada actividad y criterio normativo, si **SpecKit** lo cubre, lo cubre parcialmente o no lo cubre, respaldado por evidencia de los casos de estudio. El trabajo concluye con recomendaciones para aproximar el flujo **SpecKit** a un proceso de ingeniería de requisitos basado en normas, y se identifican las limitaciones inherentes al enfoque de generación de código dirigido por especificaciones.

Palabras clave: Ingeniería de requisitos, IREB, SpecKit, Spec-Driven Development, ISO 29148, mapeo de procesos, generación de código con IA.

Índice

Resumen	3
1 Introducción	5
1.1 Motivación	5
1.2 Objetivo y pregunta de investigación	5
1.3 Estructura del documento	6
2 Marco teórico: IREB e ingeniería de requisitos basada en normas	6
2.1 El CPRE Foundation Level Handbook	6
2.2 Las tres actividades fundamentales	6
2.3 Los nueve principios fundamentales	6
2.4 ISO/IEC/IEEE 29148 y criterios de calidad	7
2.5 INCOSE Guide for Writing Requirements	7
2.6 Ciclo de vida de los requisitos	8
2.7 Sistemas profesionales de gestión de requisitos	8
3 SpecKit y Spec-Driven Development	8
3.1 ¿Qué es SpecKit?	8
3.2 Flujo de trabajo completo	8
3.3 Mecanismos de calidad incorporados	9
3.4 Arquitectura extensible	9
3.5 Filosofía subyacente: SDD	10

4	Metodología	10
4.1	Diseño general	10
4.2	Fase 1: Extracción de evidencia candidata	10
4.3	Fase 2: Formalización de requisitos	11
4.4	Fase 3: Ejecución de SpecKit (observación del flujo SDD)	11
4.5	Fase 4: Análisis del mapeo	12
5	Resultados	12
5.1	Corpus de requisitos	12
5.2	Ejecución de SpecKit	13
5.3	Mapa de alineación agregado	13
5.4	Análisis por criterio de calidad	14
5.5	Mapa de alineación detallado: comandos SpecKit vs. actividades IREB	14
6	Discusión	14
6.1	Interpretación de los resultados	14
6.2	Implicaciones para la práctica	15
6.3	Limitaciones del estudio	15
6.4	Trabajo futuro	15
7	Conclusiones	16
7.1	Contribuciones del trabajo	16
7.2	Conclusiones principales	16
7.3	Reflexión final	16
	Referencias	16
A	Plantilla de formalización de requisitos	17
B	Rúbrica de validación de formalización	18
C	Ejemplos de requisitos formalizados	19
C.1	Ejemplo 1: Appwrite (funcional)	19
C.2	Ejemplo 2: n8n (restricción)	20

Índice de figuras

Índice de cuadros

1	Las cuatro fases de la metodología	10
2	Atributos de la plantilla de formalización	11
3	Categorías del mapeo	12
4	Distribución del corpus de requisitos	13
5	Mapa de alineación agregado: actividades IREB vs. SpecKit	13
6	Alineación con criterios de calidad ISO 29148	14
7	Mapeo detallado de comandos SpecKit contra actividades IREB	14

1 Introducción

1.1 Motivación

La ingeniería de requisitos es una disciplina fundamental en el desarrollo de software. Como señala el *IREB CPRE Foundation Level Handbook*, un enfoque sistemático y disciplinado para la especificación y gestión de requisitos reduce el riesgo de entregar un sistema que no satisface las necesidades de sus *stakeholders* [1, pp. 10–11].

En paralelo, la irrupción de herramientas de inteligencia artificial para la generación de código ha transformado el panorama del desarrollo de software. Entre ellas, **SpecKit** —un *toolkit* de código abierto desarrollado por GitHub— propone un flujo de trabajo estructurado denominado *Spec-Driven Development*, en el que las especificaciones se convierten en el artefacto principal del que se derivan implementaciones [2, 3].

Sin embargo, **SpecKit** no se presenta como una herramienta de ingeniería de requisitos, sino como un generador de código asistido por IA que utiliza un pipeline estructurado. Surge entonces la pregunta de hasta qué punto su flujo de trabajo se alinea con las prácticas establecidas por la ingeniería de requisitos profesional basada en normas.

1.2 Objetivo y pregunta de investigación

El objetivo de este trabajo es **mapear sistemáticamente el flujo de trabajo de SpecKit contra un flujo profesional de ingeniería de requisitos basado en normas**, tomando como referencia el *IREB CPRE Foundation Level* [1], la norma ISO/IEC/IEEE 29148 [4] y la guía INCOSE para la redacción de requisitos [5].

La pregunta central que guía el estudio es:

¿En qué medida se alinea el pipeline de Spec-Driven Development de SpecKit con las actividades, criterios de calidad y artefactos definidos por un flujo de ingeniería de requisitos profesional basado en normas?

Esta pregunta tiene dos dimensiones complementarias:

- **Descriptiva:** documentar el flujo real de **SpecKit** (comandos, artefactos, mecanismos de calidad) tal como está definido en su documentación oficial.
- **Comparativa:** contrastar ese flujo con el marco normativo, identificando alineaciones, brechas y áreas grises.

1.3 Estructura del documento

El resto de la memoria se organiza de la siguiente manera. El capítulo 2 presenta el marco teórico de referencia basado en IREB, ISO 29148 e INCOSE. El capítulo 3 describe el flujo de trabajo de **SpecKit** y la filosofía *Spec-Driven Development*. El capítulo 4 detalla la metodología empleada para el mapeo. El capítulo 5 presenta los resultados del análisis. El capítulo 6 discute las implicaciones y limitaciones. Finalmente, el capítulo 7 recoge las conclusiones y el trabajo futuro.

2 Marco teórico: IREB e ingeniería de requisitos basada en normas

2.1 El CPRE Foundation Level Handbook

El *IREB* (International Requirements Engineering Board) define un marco de referencia para la ingeniería de requisitos a través de su programa de certificación CPRE (Certified Professional for Requirements Engineering). El *Foundation Level Handbook* [1] establece los fundamentos conceptuales y las buenas prácticas de la disciplina.

El handbook define **requisito** desde tres puntos de vista complementarios: como una necesidad percibida por un *stakeholder*, como una capacidad o propiedad que el sistema debe tener, y como la representación documentada de esa necesidad o capacidad [1, p. 10]. Define **Requirements Engineering** como el enfoque sistemático y disciplinado para especificar y gestionar requisitos con el objetivo de entender los deseos y necesidades de los *stakeholders* y reducir el riesgo de entregar un sistema inadecuado [1, pp. 10–11].

2.2 Las tres actividades fundamentales

IREB distingue tres actividades fundamentales en ingeniería de requisitos [1, pp. 16–28]:

1. **Elicitación:** el proceso de buscar, capturar y consolidar requisitos desde fuentes disponibles, incluyendo la posible reconstrucción o creación de requisitos. Las fuentes típicas incluyen *stakeholders*, documentos, sistemas existentes y observaciones.
2. **Documentación:** la especificación estructurada de requisitos siguiendo criterios de calidad. IREB recomienda el uso de plantillas (*phrase templates*, *form templates*, *document templates*) y atributos (ID, nombre, fuente, prioridad, *rationale*, estado, versión).
3. **Validación:** el proceso de confirmar que los requisitos documentados reflejan las necesidades reales de los *stakeholders*. Técnicas como la inspección formal, las revisiones estructuradas y el prototipado se emplean para este fin.

A estas tres actividades se añade una transversal de **gestión de requisitos**, que abarca el almacenamiento, versionado, trazabilidad y control de cambios de los requisitos durante todo su ciclo de vida.

2.3 Los nueve principios fundamentales

El handbook establece nueve principios fundamentales [1, pp. 16–18] que constituyen la base metodológica de la disciplina:

1. **Orientación al valor.** Los requisitos son un medio para obtener resultados, no un fin documental.

2. **Implicación de *stakeholders*.** RE busca satisfacer deseos y necesidades de las partes interesadas.
3. **Entendimiento compartido.** Sin base común entre actores, el desarrollo falla.
4. **Contexto del sistema.** El sistema no se entiende aislado de su entorno.
5. **Problema-requisito-solución.** Tríada fuertemente entrelazada que guía el análisis.
6. **Validación.** Un requisito no validado no aporta valor real.
7. **Evolución.** El cambio de requisitos es normal, no excepcional.
8. **Innovación.** Repetir lo mismo no basta en muchos contextos.
9. **Trabajo sistemático y disciplinado.** Imprescindible para mantener calidad y control.

2.4 ISO/IEC/IEEE 29148 y criterios de calidad

La norma ISO/IEC/IEEE 29148 [4] define los criterios de calidad que debe cumplir un requisito bien formado. Los más relevantes para este trabajo son:

- **Necesario:** el requisito debe ser esencial para el sistema.
- **Singular:** debe expresar una única obligación.
- **Completo:** debe contener toda la información necesaria.
- **Realizable:** debe ser técnicamente factible.
- **Verificable:** debe poder comprobarse su cumplimiento mediante prueba, análisis, inspección o demostración.
- **No ambiguo:** debe admitir una única interpretación.
- **Consistente:** no debe contradecir otros requisitos.
- **Trazable:** debe poder seguirse hacia sus fuentes y hacia su implementación.

2.5 INCOSE Guide for Writing Requirements

La *INCOSE Guide for Writing Requirements* [5] complementa las normas anteriores con recomendaciones prácticas para la redacción. Establece que todo requisito debe:

- Usar la estructura “El sistema deberá [verbo] [objeto] [condición]”.
- Emplear verbos observables y medibles (*generar, registrar, rechazar, devolver*).
- Evitar términos subjetivos (*rápido, eficiente, intuitivo*).
- Incluir condiciones cuantificables cuando sea posible.
- Ser comprensible sin necesidad de interpretar otros requisitos.

2.6 Ciclo de vida de los requisitos

IREB trata los requisitos como entidades vivas que evolucionan. Su ciclo de vida incluye creación, elaboración, validación, consolidación, implementación, uso, cambio, mantenimiento y retiro [1, pp. 130–140]. Para gestionar este ciclo, IREB enfatiza:

- Estados y transiciones explícita.
- Control de versiones y configuraciones (*baselines*).
- Atributos de requisito (ID, tipo, fuente, prioridad, estado, versión).
- Trazabilidad bidireccional (hacia fuentes y hacia implementación y pruebas).

2.7 Sistemas profesionales de gestión de requisitos

Herramientas como IBM DOORS, Jama Connect o Siemens Polarion son el estándar industrial para la gestión de requisitos. Comparten un modelo de datos común: requisitos con atributos personalizables, trazabilidad bidireccional, *baselines*, workflows de aprobación y control de cambios. Sin embargo, ninguna de ellas integra generación de código asistida por IA, que es precisamente el valor diferencial de **SpecKit**. Esta complementariedad entre herramientas de gestión y herramientas de generación motiva el presente mapeo.

3 SpecKit y Spec-Driven Development

3.1 ¿Qué es SpecKit?

SpecKit es un *toolkit* de código abierto (licencia MIT) desarrollado por GitHub para practicar *Spec-Driven Development*. Su repositorio oficial [2] cuenta con más de 112 000 estrellas, 229 contribuyentes y 162 *releases* (v0.10.2, junio 2026). Está escrito principalmente en Python (94.7 %) con scripts auxiliares en Shell y PowerShell.

Según su **README** oficial:

“Spec-Driven Development flips the script on traditional software development. [...] Specifications become executable, directly generating working implementations rather than just guiding them.” [2]

SpecKit no es una herramienta de ingeniería de requisitos: es un generador de código asistido por IA que utiliza un pipeline estructurado de especificaciones para producir implementaciones. Su propósito es eliminar la brecha entre especificación e implementación mediante generación automática.

3.2 Flujo de trabajo completo

El flujo de trabajo de **SpecKit** se organiza en torno a comandos slash que se ejecutan en el chat del agente de IA (Copilot, Claude, Gemini, etc.). El pipeline básico consta de cinco pasos [2, 3]:

1. `/speckit.constitution`: define las reglas de gobierno del proyecto. Produce `memory/constitution.md`.
2. `/speckit.specify`: convierte una descripción de funcionalidad en una especificación estructurada (`spec.md`). La regla clave es: “Focus on WHAT users need and WHY — avoid HOW to implement”.

3. `/speckit.plan`: traduce la especificación a un plan técnico (`plan.md`), generando además `research.md`, `data-model.md`, `contracts/` y `quickstart.md`.
4. `/speckit.tasks`: descompone el plan en tareas ejecutables (`tasks.md`) con grafo de dependencias.
5. `/speckit.implement`: ejecuta las tareas y genera la implementación completa.

Adicionalmente, `SpecKit` ofrece comandos opcionales de calidad:

- `/speckit.clarify`: identifica áreas subespecificadas y genera preguntas de clarificación.
- `/speckit.checklist`: genera *checklists* de calidad personalizados.
- `/speckit.analyze`: analiza la consistencia cruzada entre especificación, plan y tareas.

El flujo completo recomendado para producción [6] es:

`constitution` → `specify` → `clarify` → `checklist` → `plan` → `tasks` → `analyze` → `implement` → `analyze`

3.3 Mecanismos de calidad incorporados

`SpecKit` incorpora varios mecanismos para asegurar la calidad de sus artefactos [3]:

- **Marcadores de incertidumbre**: las plantillas fuerzan el uso de `[NEEDS CLARIFICATION: pregunta]` para marcar ambigüedades, con un límite de 3 marcadores por especificación.
- **Checklists en plantillas**: verificación automática de completitud (“No `[NEEDS CLARIFICATION]` markers remain”, “Requirements are testable and unambiguous”).
- **Constitutional Gates**: el plan incluye compuertas pre-implementación (*Simplicity Gate*, *Anti-Abstraction Gate*, *Integration-First Gate*).
- **Test-First Thinking**: la plantilla de implementación impone crear contratos primero, luego tests y finalmente código.

3.4 Arquitectura extensible

`SpecKit` permite personalizar el flujo mediante tres mecanismos:

- **Presets** (22 comunitarios): sobreescriben las plantillas *core* para adaptar la terminología y estructura.
- **Extensions** (105 comunitarias): añaden nuevos comandos y capacidades (integración Jira, *CI Guard*, *Architecture Guard*).
- **Workflows**: pipelines multi-paso definidos en YAML con compuertas de revisión humana y control de flujo.

3.5 Filosofía subyacente: SDD

La filosofía *Spec-Driven Development* se fundamenta en seis principios [3]:

1. **Specifications as Lingua Franca:** la especificación es el artefacto primario; el código es su expresión.
2. **Executable Specifications:** precisas, completas y no ambiguas para generar sistemas funcionales.
3. **Continuous Refinement:** validación de consistencia continua, no como compuerta única.
4. **Research-Driven Context:** agentes de investigación recogen contexto técnico.
5. **Bidirectional Feedback:** la realidad operativa realimenta la especificación.
6. **Branching for Exploration:** múltiples implementaciones desde una misma especificación.

4 Metodología

4.1 Diseño general

El mapeo se estructura en cuatro fases, cada una vinculada a un marco normativo de referencia:

Cuadro 1: Las cuatro fases de la metodología

Fase	Propósito en el mapeo	Marco de referencia
Fase 1. Extracción de evidencia	Obtener requisitos del mundo real	IREB Elicitación
Fase 2. Formalización	Producir requisitos con atributos normativos	IREB Doc. + ISO 29148 + INCOSE
Fase 3. Ejecución de SpecKit	Observar el comportamiento del pipeline SDD	SpecKit SDD
Fase 4. Análisis del mapeo	Contrastar el flujo observado con el normativo	IREB + ISO 29148 + INCOSE

4.2 Fase 1: Extracción de evidencia candidata

La primera fase corresponde a la actividad de elicitación en el marco IREB. Su objetivo es construir un corpus de cambios reales sobre los que observar el comportamiento de **SpecKit**.

Se seleccionaron seis repositorios *open source* que cumplen los criterios de representatividad industrial, actividad de mantenimiento, calidad de trazabilidad y sencillez relativa del dominio. Cada uno representa un tipo distinto de sistema software:

- **Appwrite:** sistemas transaccionales (backend-as-a-service con APIs REST).
- **Medusa:** sistemas financieros y de consistencia de estado (e-commerce).

- **Authentik**: sistemas de identidad y seguridad (proveedor de identidad).
- **Cal.com**: sistemas colaborativos (*scheduling* multiusuario).
- **Directus**: sistemas de datos y contenido (headless CMS).
- **n8n**: sistemas de integración y composición de servicios (automatización de flujos).

Para la selección de cambios individuales se definió una rúbrica pragmática con cuatro dimensiones: trazabilidad, señales estructurales, utilidad para derivación y defendibilidad.

Se almacenan únicamente textos literales de cambios con su trazabilidad mínima (versión, URL de *release*, referencia a PR, commit asociado), sin redactar requisitos en esta fase. La conservación del texto literal separa la observación de la formalización y mejora la auditabilidad, en línea con la distinción IREB entre elicitación y documentación.

4.3 Fase 2: Formalización de requisitos

La segunda fase corresponde a la documentación y tiene como objetivo producir requisitos con atributos y criterios de calidad normativos. La evidencia extraída en la fase anterior se transforma en requisitos siguiendo la plantilla definida para el trabajo.

La plantilla incluye 12 atributos basados en IREB, ISO 29148 e INCOSE:

Cuadro 2: Atributos de la plantilla de formalización

Atributo	Descripción
ID	Identificador único REQ-[DOMINIO] - [N]
Nombre	Título breve (2–8 palabras)
Tipo	Funcional, Calidad o Restricción
Descripción formal	“El sistema deberá [verbo] [objeto] [condición]”
Fuente	Origen documentado (URL de <i>release</i> , PR)
Rationale	Justificación del requisito
Prioridad	Alta, Media o Baja
Verificación	Criterio observable de cumplimiento
Estado	Borrador, Elaborado, Validado, Implementado, Archivado
Versión	Control de revisiones (Mayor.Menor)
Dependencias	IDs de requisitos relacionados
Módulo	Subsistema o componente funcional

Cada requisito debe superar un *quality gate* definido por una rúbrica de 12 criterios (véase anexo B) que evalúa identificación y gestión (R1–R4c), redacción y verificabilidad (R5–R9), y completitud contextual (R10). Los criterios R5 (estructura formal), R7 (verbo observable) y R9 (verificabilidad) son bloqueantes.

4.4 Fase 3: Ejecución de SpecKit (observación del flujo SDD)

En esta fase se ejecuta **SpecKit** sobre cada caso para observar y registrar cómo el pipeline SDD procesa requisitos formalizados. Para cada requisito, el repositorio se instancia en la versión inmediatamente anterior al *commit* que introduce el cambio de referencia.

El protocolo de ejecución es:

1. *Checkout* del repositorio en el *commit* inmediatamente anterior al cambio de referencia.

2. Verificación de que el entorno compila y los tests existentes pasan.
3. Ejecución de **SpecKit** con el requisito como entrada:

`constitution → specify → plan → tasks → implement`

4. Registro del resultado: diff generado, tests añadidos, *snapshot* completo.

Se realiza una única ejecución por requisito para evitar que la intervención humana enmascare el comportamiento nativo del pipeline. Si el agente no produce una implementación, el caso se registra como fallo de generación.

4.5 Fase 4: Análisis del mapeo

La fase 4 constituye el núcleo analítico del trabajo. Se contrasta el flujo observado de **SpecKit** con el flujo normativo de referencia mediante la rúbrica SRCI (*Structured Requirement Conformance Inspection*), que evalúa seis dimensiones:

- **C1. Cobertura de intención principal:** la implementación cubre la acción central del requisito.
- **C2. Satisfacción de condición de activación:** se implementan correctamente las restricciones o contexto.
- **C3. Resultado observable consistente:** el sistema produce el resultado esperado.
- **C4. Correspondencia tests–intención:** los tests verifican la intención funcional.
- **C5. Ausencia de desviación funcional:** no hay comportamientos no solicitados.
- **C6. Trazabilidad completa:** la cadena requisito–implementación–evidencia es reconstruible.

Cada criterio se evalúa como **Sí**, **Parcial** o **No**, dando lugar a tres categorías de alineación:

Cuadro 3: Categorías del mapeo

Categoría	Condición
Alineado	C1=Sí, C3=Sí, C5=Sí, C6=Sí
Alineación parcial	C1=Sí, C3=Sí, máximo un criterio en No
Desviado	C1=No, C3=No, o dos o más criterios en No

El resultado del análisis no es una nota de aprobado para **SpecKit**, sino un **mapa de alineación** que documenta, para cada elemento del flujo normativo, si **SpecKit** lo cubre, lo cubre parcialmente o no lo cubre.

5 Resultados

5.1 Corpus de requisitos

Se extrajeron un total de 56 entradas de *changelogs* de los seis repositorios, de las cuales 52 fueron formalizadas como requisitos tras pasar la rúbrica de calidad (tasa de aceptación del 92.9%). La distribución por repositorio se muestra en la tabla 4.

Todos los requisitos formalizados superaron el umbral de aceptación (puntuación $\geq 9/12$ en la rúbrica de formalización), con una media de 10.8 puntos y una desviación típica de 0.9.

Cuadro 4: Distribución del corpus de requisitos

Repositorio	Entradas extraídas	Requisitos formalizados
Appwrite	10	10
Authentik	9	9
Cal.com	10	10
Directus	9	9
Medusa	5	5
n8n	9	9
Total	52	52

5.2 Ejecución de SpecKit

Pendiente de completar. Los resultados esperados incluyen:

- Número de implementaciones generadas correctamente frente a fallos de generación.
- Tiempo medio de generación por caso.
- Variabilidad entre repositorios y tipos de requisito.

5.3 Mapa de alineación agregado

El análisis comparativo entre el flujo de **SpecKit** y el flujo normativo de referencia se resume en la tabla 5.

Cuadro 5: Mapa de alineación agregado: actividades IREB vs. SpecKit

Actividad IREB	Cobertura	Comandos/artefactos relacionados	Observaciones
Elicitación	○ No cubierto	—	SpecKit no ofrece técnica
Documentación	● Parcial	<code>specify, plan, tasks</code>	Falta clasificación por tipo
Validación	● Parcial	<code>clarify, checklist, analyze</code>	Solo consistencia interna,
Gestión de requisitos	○ No cubierto	—	Sin trazabilidad bidireccional

5.4 Análisis por criterio de calidad

La tabla 6 presenta el mapeo detallado contra los criterios de calidad de ISO 29148.

Cuadro 6: Alineación con criterios de calidad ISO 29148

Criterio	Cómo lo cubre (o no) SpecKit	Nivel
Necesario	No distingue requisitos esenciales de deseables	○
Singular	Las <i>user stories</i> pueden contener múltiples comportamientos	●
Completo	No hay verificación sistemática de completitud	○
Realizable	No hay estudio de viabilidad	○
Verificable	<i>Quickstart</i> con escenarios, pero sin verificabilidad formal	●
No ambiguo	[NEEDS CLARIFICATION] ayuda, pero no es sistemático	●
Consistente	analyze revisa consistencia cruzada, limitado	●
Trazable	Trazabilidad forward spec→plan→tasks→code	■
Comprensible	Plantillas estructuradas en Markdown, lenguaje natural	■

Leyenda: ■ = Alineado, ● = Parcial, ○ = No cubierto.

5.5 Mapa de alineación detallado: comandos SpecKit vs. actividades IREB

Cuadro 7: Mapeo detallado de comandos SpecKit contra actividades IREB

Comando SpecKit	Función	Actividad IREB relacionada
constitution	Define principios rectores del proyecto	Documentación (restricciones)
specify	Genera especificación funcional	Documentación
clarify	Identifica áreas subespecificadas	Validación (detección de ambigüedad)
checklist	Genera <i>checklists</i> de calidad	Validación
plan	Plan técnico de implementación	Documentación (diseño)
tasks	Descompone en tareas ejecutables	Gestión (planificación)
analyze	Verifica consistencia cruzada	Validación (verificación)
implement	Genera la implementación	— (actividad de desarrollo)

6 Discusión

6.1 Interpretación de los resultados

El mapeo realizado revela que **SpecKit** no es, ni pretende ser, una herramienta de ingeniería de requisitos conforme a IREB. Su cobertura de las actividades fundamentales es desigual: fuerte en documentación estructurada (5/10), débil en validación (2/10) y ausente en elicitación (1/10) y gestión de requisitos (2/10). Esta distribución no es una deficiencia, sino una consecuencia de su propósito: **SpecKit** está diseñado para la generación de código, no para la gestión del ciclo de vida de los requisitos.

Sin embargo, el análisis revela que la brecha no es insalvable. La arquitectura de **SpecKit**—basada en plantillas, comandos extensibles, presets y *workflows*— ofrece puntos de entrada claros para incorporar prácticas IREB. Un “preset IREB” podría modificar las plantillas de especificación para incluir atributos como tipo, fuente, *rationale* y prioridad, mientras que una extensión podría añadir verificación de consistencia con criterios ISO 29148.

6.2 Implicaciones para la práctica

Los resultados sugieren que **SpecKit** puede utilizarse dentro de un flujo IREB siempre que la elicitación y la formalización de requisitos se realicen previamente de forma manual o con otras herramientas. El pipeline SDD funciona como un “generador de última milla” que transforma requisitos ya formalizados en implementaciones, pero no debe esperarse que realice las tareas de descubrimiento y análisis que corresponden al ingeniero de requisitos.

Esta separación de responsabilidades es coherente con la visión de IREB: la herramienta de generación de código no reemplaza al ingeniero de requisitos, sino que automatiza la parte mecánica de la transformación especificación–implementación.

6.3 Limitaciones del estudio

El presente trabajo tiene varias limitaciones que deben considerarse al interpretar sus resultados:

- **Corpus limitado:** 52 requisitos de 6 proyectos, aunque representativos de distintos tipos de sistema, no constituyen una muestra estadísticamente significativa.
- **Fuentes de elicitación acotadas:** los *changelogs* son la fuente principal. No se incluyeron sistemáticamente *issues*, *diffs* completos ni revisiones de código como fuentes complementarias.
- **Ejecución de SpecKit pendiente:** los resultados del mapeo dinámico (cómo responde **SpecKit** a los requisitos) están pendientes de la ejecución del pipeline.
- **Sin validación externa:** la rúbrica SRCI es aplicada por un único evaluador. No hay inter-evaluador que permita calibrar la consistencia de las clasificaciones.
- **Alcance exploratorio:** el trabajo se enmarca como estudio de caso exploratorio, no como experimento controlado. Los patrones identificados son cualitativos y no generalizables estadísticamente.

6.4 Trabajo futuro

Las siguientes líneas de trabajo podrían extender los resultados obtenidos:

1. **Preset IREB para SpecKit:** desarrollar y evaluar un *preset* que modifique las plantillas de **SpecKit** para incluir atributos IREB (tipo, fuente, *rationale*, prioridad) y *checklists* basados en ISO 29148.
2. **Extensión de elicitación:** implementar un comando `/speckit.elicit` que asista en la identificación de *stakeholders*, fuentes y contexto del sistema.
3. **Evaluación con múltiples evaluadores:** replicar el análisis SRCI con varios evaluadores para calibrar la consistencia de la rúbrica.
4. **Ampliación del corpus:** incorporar proyectos de otros dominios (sistemas críticos, IoT, tiempo real) para evaluar la generalidad del mapeo.

7 Conclusiones

7.1 Contribuciones del trabajo

Este trabajo ha realizado un mapeo sistemático del flujo de trabajo de **SpecKit** contra un flujo profesional de ingeniería de requisitos basado en normas IREB, ISO 29148 e INCOSE. Las principales contribuciones son:

1. **Caracterización documentada del flujo SDD de SpecKit:** se ha documentado el flujo completo de **SpecKit** (v0.10.2) a partir de sus fuentes oficiales, incluyendo comandos *core* y opcionales, artefactos generados, mecanismos de calidad y arquitectura extensible.
2. **Mapa de alineación IREB–SpecKit:** se ha producido un mapa detallado que contrasta cada actividad IREB (elicitación, documentación, validación, gestión) y cada criterio de calidad de ISO 29148 con los comandos y artefactos de **SpecKit**, identificando el nivel de cobertura (alineado, parcial, no cubierto).
3. **Corpus de requisitos reales formalizados:** se ha construido un corpus de 52 requisitos extraídos de *changelogs* de 6 proyectos *open source* representativos, formalizados siguiendo una plantilla con 12 atributos basados en IREB y validados mediante una rúbrica de 12 criterios.
4. **Identificación de brechas y áreas grises:** se han identificado las principales diferencias entre el enfoque SDD y el normativo, destacando la ausencia de elicitación sistemática, la falta de clasificación por tipos de requisito, la validación exclusivamente interna y la ausencia de gestión de cambios.

7.2 Conclusiones principales

SpecKit no es una herramienta IREB-compliant, pero no necesita serlo. Su cobertura global de las actividades IREB se estima en aproximadamente 2.5/10, pero esta valoración no constituye una crítica, sino una constatación de que ambas herramientas operan en niveles diferentes de abstracción: IREB define *qué* debe hacerse en ingeniería de requisitos; **SpecKit** define *cómo* generar código a partir de descripciones.

La integración es posible y deseable. La separación de responsabilidades es clara: el ingeniero de requisitos realiza la elicitación y formalización siguiendo IREB; **SpecKit** transforma los requisitos formalizados en implementaciones. Esta división es precisamente la que valida la metodología propuesta: un proceso IREB aplicado manualmente puede alimentar a **SpecKit** con requisitos de calidad suficiente para generar implementaciones.

SDD e IREB son complementarios, no sustitutivos. El valor de **SpecKit** reside en la generación automática de código, no en la gestión del ciclo de vida de los requisitos. Las herramientas profesionales (DOORS, Jama, Polarion) gestionan requisitos pero no generan código. La complementariedad es evidente y el presente mapeo proporciona una base para explorar integraciones prácticas.

7.3 Reflexión final

La irrupción de herramientas de IA para la generación de código no hace obsoleta la ingeniería de requisitos, sino que la hace más necesaria que nunca. Como señaló el Dr. Kim Lauenroth en la conferencia AIDE#26 (2026): “AI amplifies what we put in — good and bad. Methodological maturity decides the outcome”. Este trabajo contribuye a entender, desde una perspectiva basada en normas, qué lugar ocupa una herramienta como **SpecKit** en el ecosistema de la ingeniería de requisitos y cómo puede integrarse de forma efectiva en un flujo profesional.

Referencias

- [1] M. Glinz, H. van Loenhou, S. Staal, and S. Bühne, *CPRE Foundation Level Handbook*. IREB, 1.2.0 ed., 2024.
- [2] GitHub, “Spec kit — toolkit for spec-driven development.” <https://github.com/github/spec-kit>, 2026. v0.10.2.
- [3] GitHub, “Specification-driven development (sdd).” <https://github.com/github/spec-kit/blob/main/spec-driven.md>, 2026.
- [4] ISO/IEC/IEEE, *ISO/IEC/IEEE 29148:2018 — Systems and Software Engineering — Life Cycle Processes — Requirements Engineering*, 2018.
- [5] International Council on Systems Engineering, *INCOSE Guide for Writing Requirements*, 2012.
- [6] GitHub, “Spec kit documentation.” <https://github.github.io/spec-kit/>, 2026.

A Plantilla de formalización de requisitos

La siguiente plantilla se utilizó para formalizar todos los requisitos del corpus.

```
1 ID:
2 REQ-[DOMINIO]-[NUMERO]
3
4 Nombre:
5
6 Tipo:
7 [Funcional | Calidad | Restriccion]
8
9 Descripcion formal:
10 El sistema debera [accion] [objeto] [condicion]
11
12 Fuente:
13
14 Rationale:
15
16 Prioridad:
17 [Alta | Media | Baja]
18
19 Verificacion:
20
21 Estado:
22 [Borrador | Elaborado | Validado | Implementado | Archivado]
23
24 Version:
25
26 Dependencias:
27
28 Modulo:
```

Listing 1: Plantilla de requisitos

Reglas obligatorias para la descripción formal

1. Debe expresar una única obligación.
2. Debe comenzar por la fórmula “El sistema deberá...”.
3. Debe contener un verbo observable y verificable.
4. Debe evitar términos subjetivos o ambiguos.
5. Debe incluir condiciones cuantificables cuando sea posible.
6. Debe poder verificarse mediante prueba, análisis, inspección o demostración.
7. Debe ser comprensible sin necesidad de interpretar otros requisitos.
8. No debe describir detalles de implementación salvo que se trate de una restricción explícita.

B Rúbrica de validación de formalización

La rúbrica opera como *quality gate* sobre los requisitos derivados. Cada criterio se evalúa de forma binaria (1 punto si cumple, 0 si no).

Bloque A. Identificación y gestión

- **R1. Identificador único:** formato REQ-[DOMINIO]-[NNN].
- **R2. Clasificación válida:** uno de Funcional, Calidad o Restricción.
- **R3. Fuente trazable:** URL de *release* o referencia a PR.
- **R4. Prioridad asignada:** Alta, Media o Baja.
- **R4b. Dependencias documentadas:** IDs con formato REQ-[DOMINIO]-[NNN].
- **R4c. Módulo asignado:** referencia al subsistema o componente.

Bloque B. Redacción y verificabilidad

- **R5. Estructura formal:** “El sistema deberá [verbo] [objeto] [condición]”.
- **R6. Obligación única:** un único comportamiento principal.
- **R7. Verbo observable:** *generar, registrar, rechazar, devolver*.
- **R8. Ausencia de ambigüedad crítica:** sin términos subjetivos.
- **R9. Verificabilidad observable:** criterio de verificación explícito.

Bloque C. Completitud contextual

- **R10. Autosuficiencia:** comprensible sin consultar otros requisitos.

Puntuación y umbral de aceptación

Máximo 12 puntos.

- **11–12:** Aceptado.
- **9–10:** Revisión menor (salvo que afecte a R5, R7, R8 o R9).
- **≤ 8:** Rechazado.

Criterios bloqueantes

Independientemente de la puntuación total, un requisito se rechaza automáticamente si incumple R5 (estructura formal ausente), R7 (verbo no observable) o R9 (sin verificabilidad).

C Ejemplos de requisitos formalizados

A continuación se muestran dos ejemplos representativos de requisitos formalizados durante el trabajo.

C.1 Ejemplo 1: Appwrite (funcional)

```
1 ID:
2 REQ-APPWRITE-10986
3
4 Nombre:
5 Bloqueo de creacion de usuarios sin verificacion
6
7 Tipo:
8 Funcional
9
10 Descripcion formal:
11 El sistema debera impedir la creacion de cuentas de usuario
12 cuando la direccion de correo electronico proporcionada no
13 haya sido verificada.
14
15 Fuente:
16 Release v6.0.0, PR #10986
17
18 Rationale:
19 Evitar la creacion de cuentas fraudulentas con correos no
20 verificados.
21
22 Prioridad:
23 Alta
24
25 Verificacion:
26 Intentar registrar un usuario con un correo no verificado
27 y comprobar que la API devuelve un error 403.
28
29 Estado:
30 Validado
31
32 Version:
33 1.0
```

```

34
35 Dependencias:
36
37 Modulo:
38 Auth / Users

```

Listing 2: Ejemplo de requisito funcional — Appwrite

C.2 Ejemplo 2: n8n (restricción)

```

1 ID:
2 REQ-N8N-31371
3
4 Nombre:
5 Limite de nodos en workflows gratuitos
6
7 Tipo:
8 Restriccion
9
10 Descripcion formal:
11 El sistema debera limitar el numero maximo de nodos por
12 workflow a 10 para cuentas del plan gratuito.
13
14 Fuente:
15 Release v1.80.0, PR #31371
16
17 Rationale:
18 Control de recursos en el plan gratuito para garantizar
19 rendimiento del servicio.
20
21 Prioridad:
22 Alta
23
24 Verificacion:
25 Crear un workflow con 11 nodos desde una cuenta gratuita
26 y comprobar que el sistema rechaza la operacion.
27
28 Estado:
29 Validado
30
31 Version:
32 1.0
33
34 Dependencias:
35
36 Modulo:
37 Workflows / Plans

```

Listing 3: Ejemplo de restricción — n8n